

10 High-Radix Multipliers

Chapter Goals

Study techniques that allow us to handle more than one multiplier bit in each cycle (two bits in radix 4, three in radix 8, . . .)

Chapter Highlights

- High radix gives rise to “difficult” multiples
- Recoding (change of digit-set) as remedy
- Carry-save addition reduces cycle time
- Implementation and optimization methods

High-Radix Multipliers: Topics

Topics in This Chapter

10.1 Radix-4 Multiplication

10.2 Modified Booth's Recoding

10.3 Using Carry-Save Adders

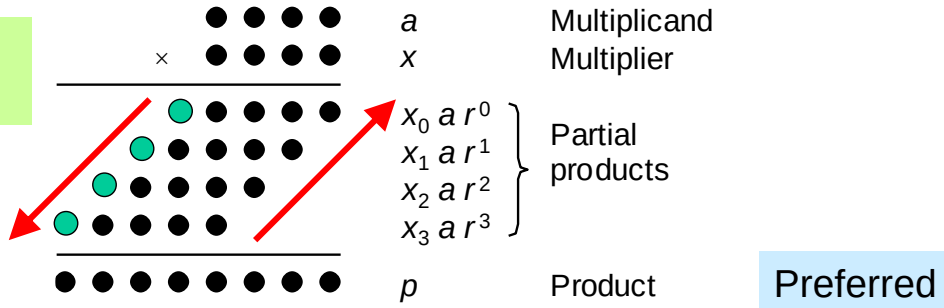
10.4 Radix-8 and Radix-16 Multipliers

10.5 Multibeat Multipliers

10.6 VLSI Complexity Issues

10.1 Radix-4 Multiplication

Fig. 9.1
(modified)



Multiplication with right shifts in radix r : top-to-bottom accumulation

$$p^{(j+1)} = (p^{(j)} + x_j a r^k) r^{-1} \quad \text{with} \quad p^{(0)} = 0 \quad \text{and} \\ p^{(k)} = p = ax + p^{(0)} r^{-k}$$

Multiplication with left shifts in radix r : bottom-to-top accumulation

$$p^{(j+1)} = \underset{\substack{\text{shift} \\ \text{---add---}}}{rp^{(j)} + x_{k-j-1}a} \quad \text{with} \quad p^{(0)} = 0 \quad \text{and} \quad p^{(k)} = p = ax + p^{(0)}r^k$$

Slide 31

Radix-4 Multiplication in Dot Notation

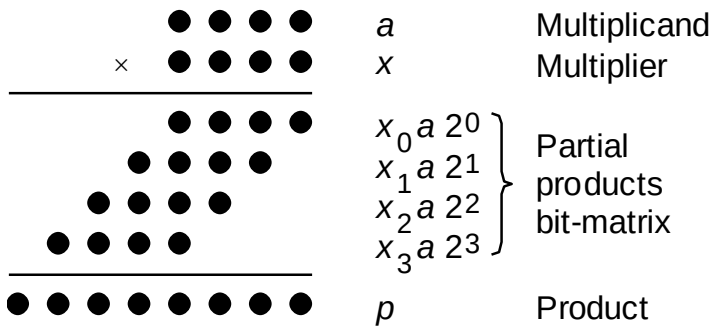
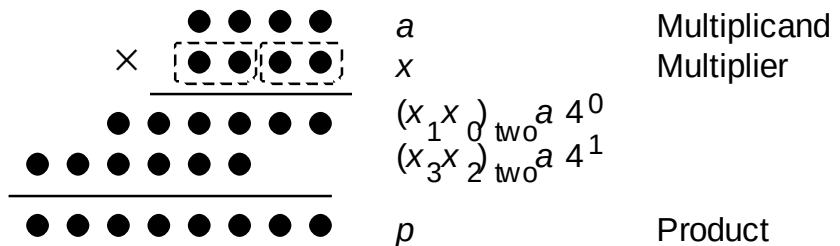


Fig. 9.1

Fig. 10.1 Radix-4, or two-bit-at-a-time, multiplication in dot notation

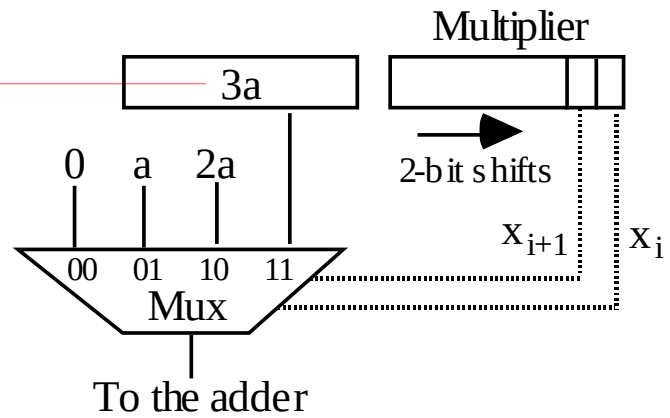
Number of cycles is halved, but now the “difficult” multiple $3a$ must be dealt with



Slide 32

A Possible Design for a Radix-4 Multiplier

Precomputed via
shift-and-add
($3a = 2a + a$)



$k/2 + 1$ cycles, rather than k

One extra cycle over $k/2$
not too bad, but we would like
to avoid it if possible

Solving this problem for radix 4
may also help when dealing
with even higher radices

Fig. 10.2 The multiple generation part of a radix-4 multiplier with precomputation of $3a$.

Example Radix-4 Multiplication Using $3a$

=====									
a				0	1	1	0		
$3a$			0	1	0	0	1	0	
x				1	1	1	0		
=====									
$p^{(0)}$				0	0	0	0		
$+(x_1x_0)_{\text{two}}a$			0	0	1	1	0	0	

$4p^{(1)}$			0	0	1	1	0	0	
$p^{(1)}$				0	0	1	1	0	0
$+(x_3x_2)_{\text{two}}a$		0	1	0	0	1	0		

$4p^{(2)}$		0	1	0	1	0	1	0	0
$p^{(2)}$			0	1	0	1	0	1	0
=====									

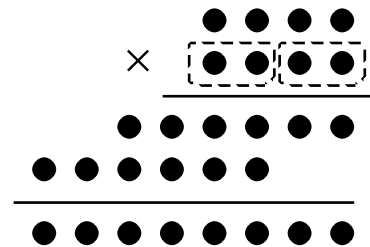
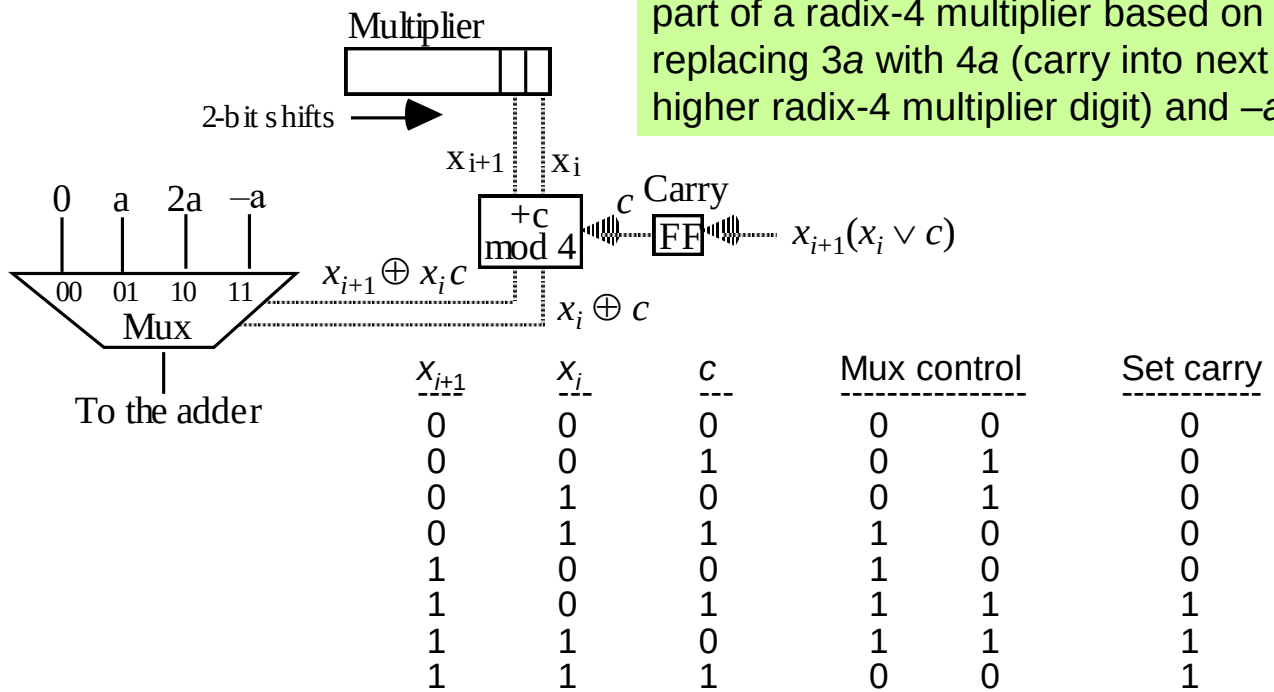


Fig. 10.3 Example of radix-4 multiplication using the $3a$ multiple.

A Second Design for a Radix-4 Multiplier

Fig. 10.4 The multiple generation part of a radix-4 multiplier based on replacing $3a$ with $4a$ (carry into next higher radix-4 multiplier digit) and $-a$.



13.1 Shift/Subtract Division Algorithms

Notation for our discussion of division algorithms:

z	Dividend	$z_{2k-1}z_{2k-2} \dots z_3z_2z_1z_0$
d	Divisor	$d_{k-1}d_{k-2} \dots d_1d_0$
q	Quotient	$q_{k-1}q_{k-2} \dots q_1q_0$
s	Remainder, $z - (d \times q)$	$s_{k-1}s_{k-2} \dots s_1s_0$

Initially, we assume unsigned operands

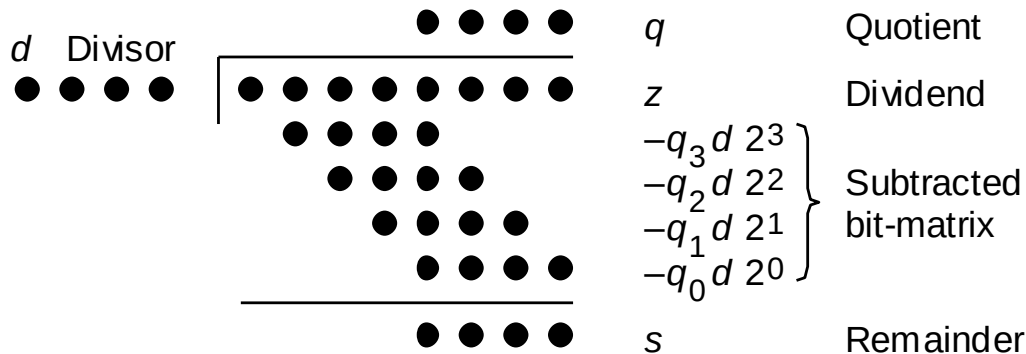


Fig. 13.1 Division of an 8-bit number by a 4-bit number in dot notation.

May 2020



Computer Arithmetic, Division

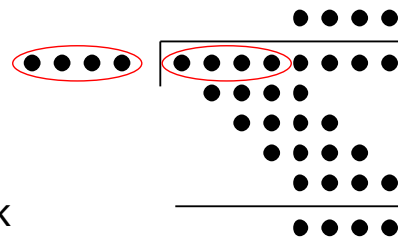


Slide 7

Division versus Multiplication

Division is more complex than multiplication:
Need for quotient digit selection or estimation

Overflow possibility: the high-order k bits of z
must be strictly less than d ; this overflow check
also detects the divide-by-zero condition.



Pentium III latencies

Instruction	Latency	Cycles/Issue
Load / Store	3	1
Integer Multiply	4	1
Integer Divide	36	36
Double/Single FP Multiply	5	2
Double/Single FP Add	3	1
Double/Single FP Divide	38	38

The ratios haven't changed much in later Pentiums, Atom, or AMD products*

*Source: T. Granlund, "Instruction Latencies and Throughput for AMD and Intel x86 Processors," Feb. 2012

May 2020



Computer Arithmetic, Division



Slide 8

Fig. 13.1

Division with left shifts (There is no corresponding right-shift algorithm)

with $s^{(0)} = z$ and
 $s^{(k)} = 2^k s$

No-overflow condition for fractions is:

$$z_{\text{frac}} < d_{\text{frac}}$$



BParhami

Examples of Basic Division

z_{frac}	. 0 1 1 1	0 1 0 1
d_{frac}	. 1 0 1 0	
$s^{(0)}$	. 0 1 1 1	0 1 0 1
$2s^{(0)}$	0 . 1 1 1 0	1 0 1
$-q_{-1}d$	. 1 0 1 0	$\{q_{-1}=1\}$
$s^{(1)}$	. 0 1 0 0	1 0 1
$2s^{(1)}$	0 . 1 0 0 1	0 1
$-q_{-2}d$	. 0 0 0 0	$\{q_{-2}=0\}$
$s^{(2)}$	. 1 0 0 1	0 1
$2s^{(2)}$	1 . 0 0 1 0	1
$-q_{-3}d$	. 1 0 1 0	$\{q_{-3}=1\}$
$s^{(3)}$	. 1 0 0 0	1
$2s^{(3)}$	1 . 0 0 0 1	
$-q_{-4}d$	. 1 0 1 0	$\{q_{-4}=1\}$
$s^{(4)}$	. 0 1 1 1	
s_{frac}	0 . 0 0 0 0	0 1 1 1
q_{frac}	. 1 0 1 1	



BParham

Slide 10

Numerical Examples for Division by 5

Instruction sequence for division by 5

$q \leftarrow z + z$ shift-left 1	{3z computed}
$q \leftarrow q + q$ shift-right 4	{ $3z(1+2^{-4})$ computed}
$q \leftarrow q + q$ shift-right 8	{ $3z(1+2^{-4})(1+2^{-8})$ computed}
$q \leftarrow q + q$ shift-right 16	{ $3z(1+2^{-4})(1+2^{-8})(1+2^{-16})$ computed}
$q \leftarrow q$ shift-right 4	{ $3z(1+2^{-4})(1+2^{-8})(1+2^{-16})/16$ computed}

Computing $29 \div 5$ ($z = 29, d = 5$)

```

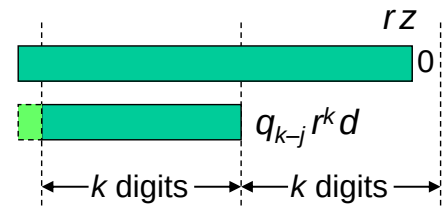
87 ← 29 + 29 shift-left 1    {3z computed}
92 ← 87 + 87 shift-right 4    {3z(1+2-4) computed}
92 ← 92 + 92 shift-right 8    {3z(1+2-4)(1+2-8) computed}
92 ← 92 + 92 shift-right 16   {3z(1+2-4)(1+2-8)(1+2-16) computed}
 5 ← 92 shift-right 4         {3z(1+2-4)(1+2-8)(1+2-16)/16 computed}

```

Repeat the process for computing $30 \div 5$ and comment on the outcome

14.1 Basics of High-Radix Division

Radices of practical interest are powers of 2, and perhaps 10



Division with left shifts

$$s^{(j)} = \underset{\substack{\text{--shift--} \\ \text{--subtract--}}}{rs^{(j-1)} - q_{k-j}(r^k d)}$$

with $s^{(0)} = z$ and
 $s^{(k)} = r^k s$

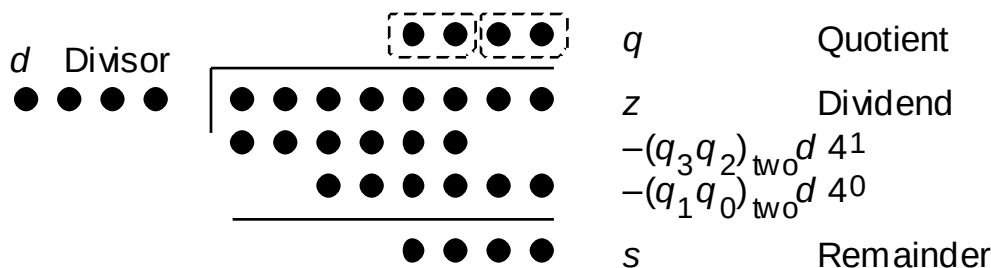


Fig. 14.1
Radix-4
division in
dot notation

Difficulty of Quotient Digit Selection

What is the first quotient digit in the following radix-10 division?

2043 | 12257968

$$12 / 2 = 6$$

$$122 / 20 = 6$$

$$1225 / 204 = 6$$

$$12257 / 2043 = 5$$

The problem with the pencil-and-paper division algorithm is that there is no room for error in choosing the next quotient digit

In the worst case, all k digits of the divisor and $k + 1$ digits in the partial remainder are needed to make a correct choice

Suppose we used the redundant signed digit set $[-9, 9]$ in radix 10

Then, we could choose 6 as the next quotient digit, knowing that we can recover from an incorrect choice by using negative digits: $59 = 6 \cdot 1$

May 2020



Computer Arithmetic, Division



Slide 38

Examples of High-Radix Division

Radix-4 integer division

z	0	1	2	3	1	1	2	3
4^4d	1	2	0	3				
$S^{(0)}$	0	1	2	3	1	1	2	3
$4S^{(0)}$	0	1	2	3	1	1	2	3
$-q_3 4^4d$	0	1	2	0	3			$\{q_3 = 1\}$
$S^{(1)}$	0	0	2	2	1	2	3	
$4S^{(1)}$	0	0	2	2	1	2	3	
$-q_2 4^4d$	0	0	0	0	0			$\{q_2 = 0\}$
$S^{(2)}$	0	2	2	1	2	3		
$4S^{(2)}$	0	2	2	1	2	3		
$-q_1 4^4d$	0	1	2	0	3			$\{q_1 = 1\}$
$S^{(3)}$	1	0	0	3	3			
$4S^{(3)}$	1	0	0	3	3			
$-q_0 4^4d$	0	3	0	1	2			$\{q_0 = 2\}$
$S^{(4)}$	1	0	2	1				
S					1	0	2	1
q					1	0	1	2

Radix-10 fractional division

$$\begin{array}{rcccc}
z_{\text{frac}} & . & 7 & 0 & 0 & 3 \\
d_{\text{frac}} & . & 9 & 9 & & \\
\hline
s^{(0)} & . & 7 & 0 & 0 & 3 \\
10s^{(0)} & 7 & . & 0 & 0 & 3 \\
-q_{-1}d & 6 & . & 9 & 3 & \{q_{-1} = 7\} \\
\hline
s^{(1)} & . & 0 & 7 & 3 & \\
10s^{(1)} & 0 & . & 7 & 3 & \\
-q_{-2}d & 0 & . & 0 & 0 & \{q_{-2} = 0\} \\
\hline
s^{(2)} & . & 7 & 3 & & \\
s_{\text{frac}} & . & 0 & 0 & 7 & 3 \\
q_{\text{frac}} & . & 7 & 0 & & \\
\hline
\end{array}$$

Fig. 14.2 Examples of high-radix division with integer and fractional operands.

May 2020



Computer Arithmetic, Division



Slide 39

Requirements for Arithmetic

Results of the 4 basic arithmetic operations (+, −, ×, ÷) as well as square-rooting must match those obtained if all intermediate computations were infinitely precise

That is, a floating-point arithmetic operation should introduce no more imprecision than the error attributable to the final rounding of a result that has no exact representation (this is the best possible)

Example:

$$\begin{array}{lcl} & (1 + 2^{-1}) & \times & (1 + 2^{-23}) \\ & \swarrow & \downarrow & \searrow \\ \text{Exact result} & 1 + 2^{-1} + 2^{-23} + 2^{-24} & & \\ \text{Chopped result} & 1 + 2^{-1} + 2^{-23} & & \text{Error} = -\frac{1}{2} \text{ ulp} \\ \text{Rounded result} & 1 + 2^{-1} + 2^{-22} & & \text{Error} = +\frac{1}{2} \text{ ulp} \end{array}$$

May 2015



Computer Arithmetic, Real Arithmetic



Slide 15

17.3 Basic Floating-Point Algorithms

Addition

Assume $e_1 \geq e_2$; *alignment shift (preshift)* is needed if $e_1 > e_2$

$$\begin{aligned} (\pm s_1 \times b^{e_1}) + (\pm s_2 \times b^{e_2}) &= (\pm s_1 \times b^{e_1}) + (\pm s_2 / b^{e_1 - e_2}) \times b^{e_1} \\ &= (\pm s_1 \pm s_2 / b^{e_1 - e_2}) \times b^{e_1} = \pm s \times b^e \end{aligned}$$

Example:

Numbers to be added:

$$x = 2^5 \times 1.00101101$$

$$y = 2^1 \times 1.11101101$$

Operand with
smaller exponent
to be preshifted

Operands after alignment shift:

$$x = 2^5 \times 1.00101101$$

$$y = 2^5 \times 0.000111101101$$

Result of addition:

$$s = 2^5 \times 1.010010111101$$

$$s = 2^5 \times 1.01001100$$

Extra bits to be
rounded off

Rounded sum

Rounding,
overflow,
and
underflow
issues
discussed
later

May 2015



Computer Arithmetic, Real Arithmetic



Slide 16

Floating-Point Multiplication and Division

Multiplication

$$(\pm s_1 \times b^{e_1}) \times (\pm s_2 \times b^{e_2}) = (\pm s_1 \times s_2) \times b^{e_1+e_2}$$

Because $s_1 \times s_2 \in [1, 4)$, postshifting may be needed for normalization

Overflow or underflow can occur during multiplication or normalization

Division

$$(\pm s_1 \times b^{e_1}) / (\pm s_2 \times b^{e_2}) = (\pm s_1/s_2) \times b^{e_1-e_2}$$

Because $s_1/s_2 \in (0.5, 2)$, postshifting may be needed for normalization

Overflow or underflow can occur during division or normalization

Floating-Point Square-Rooting

For e even: $\sqrt{s \times b^e} = \sqrt{s} \times b^{e/2}$

For e odd: $\sqrt{bs \times b^{e-1}} = \sqrt{bs} \times b^{(e-1)/2}$

After the adjustment of s to bs and e to $e - 1$, if needed, we have:

$$\sqrt{s^* \times b^{e^*}} = \sqrt{s^*} \times b^{e^*/2}$$

Even

In $[1, 4)$ for IEEE 754 In $[1, 2)$ for IEEE 754

Overflow or underflow is impossible; no postnormalization needed

17.4 Conversions and Exceptions

Conversions from fixed- to floating-point

Conversions between floating-point formats

Conversion from high to lower precision: Rounding

The IEEE 754-2008 standard includes five rounding modes:

Round to nearest, ties away from 0 (rtna)

Round to nearest, ties to even (rtne) [default rounding mode]

Round toward zero (inward)

Round toward $+\infty$ (upward)

Round toward $-\infty$ (downward)

Exceptions in Floating-Point Arithmetic

Divide by zero

Overflow

Underflow

Inexact result: Rounded value not the same as original

Invalid operation: examples include

Addition	$(+\infty) + (-\infty)$
----------	-------------------------

Multiplication $0 \times \infty$

Division $0/0$ or ∞/∞

Square-rooting operand < 0

Produce NaN as their results

17.6 Logarithmic Number Systems

Sign-and-logarithm number system: Limiting case of FLP representation

$$x = \pm b^e \times 1$$

$$e = \log_b |x|$$

We usually call b the logarithm base, not exponent base

Using an integer-valued e wouldn't be very useful, so we consider e to be a fixed-point number

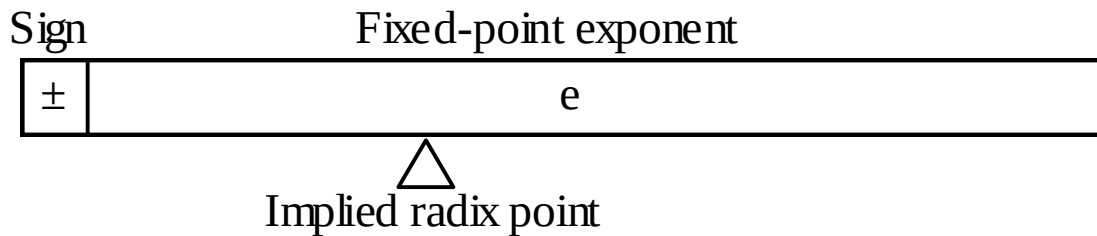


Fig. 17.13 Logarithmic number representation with sign and fixed-point exponent.

Properties of Logarithmic Representation

The logarithm is often represented as a 2's-complement number

$$(Sx, Lx) = (\text{sign}(x), \log_2 |x|)$$

Simple multiplication and division; harder add and subtract

$$L(xy) = Lx + Ly$$

$$L(x/y) = Lx - Ly$$

Example: 12-bit, base-2, logarithmic number system

$$1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1$$

Sign

Radix point

The bit string above represents $-2^{-9.828125} \cong -(0.0011)_{\text{ten}}$

Number range $\cong (-2^{16}, 2^{16})$; $min = 2^{-16}$

Advantages of Logarithmic Representation

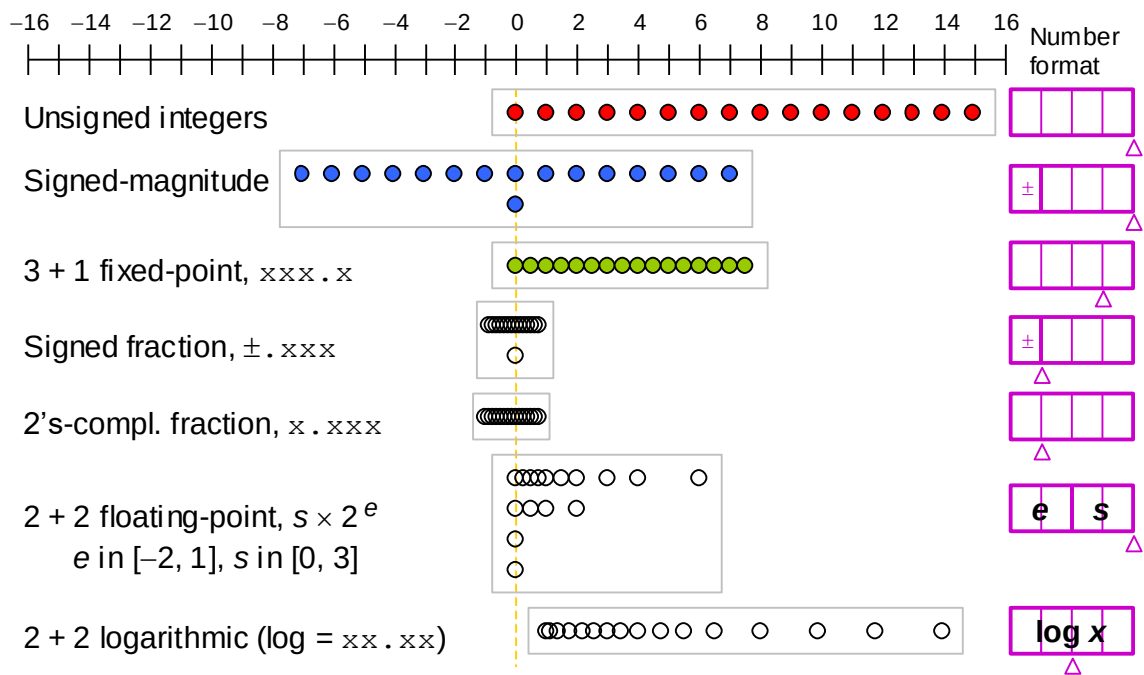


Fig. 1.2 Some of the possible ways of assigning 16 distinct codes to represent numbers.